МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ З ЛАБОРАТОРНОЇ
№6
на тему “Замикання та декоратори як HOF”
Варіант 15

Виконала:
Студентка 2 курсу
групи ФЕП-22
Сулятицька Христина
Перевірив:
Аспірант
Остріков Данило Андрійович

Львів-2026

Мета роботи.

Ознайомитися із замиканнями (closures) та функціями вищого порядку (HOF), навчитися створювати параметризовані декоратори й застосовувати їх композицію. Реалізувати декоратори таймінгу, повторних спроб і кешування з інвалідацією, використати замикання-логер, дослідити вплив порядку декораторів на виконання сервісної функції та кеш-хіти. Створити детерміновані тести pytest з ін'єкцією залежностей.

Теоретичні відомості:

Замикання (closure) — це функція, яка зберігає доступ до змінних із зовнішньої області видимості навіть після завершення виконання зовнішньої функції. Замикання дає змогу зберігати стан без створення окремого класу. Для зміни змінної зовнішньої функції у внутрішній функції застосовується оператор nonlocal.

Функція вищого порядку (HOF) приймає іншу функцію як аргумент або повертає функцію як результат. Декоратор є функцією, яка приймає функцію та повертає її обгорнуту версію. Параметризований декоратор реалізується через фабрику: спочатку викликається функція з налаштуваннями, яка повертає власне декоратор.

Декоратор timed вимірює тривалість виконання функції. Декоратор retries повторює виклик після визначених винятків, використовуючи затримку та експоненційний backoff. Декоратор cached зберігає результати за ключами, сформованими з аргументів, підтримує час життя TTL і видалення записів. Декоратор functools.wraps зберігає метадані обгорнутої функції, зокрема її ім'я, документацію та посилання __wrapped__.

Для типізації декораторів застосовуються ParamSpec і TypeVar, які дають змогу зберегти типи аргументів і результату. Залежності clock, sleep, logger та time_fn передаються як Callable. Це робить поведінку декораторів контрольованою та дає змогу тестувати їх без реального очікування або нестабільного вимірювання часу.

Хід роботи

Для 15-го варіанта реалізовано композицію декораторів для сервісної функції: кешування, повторні спроби та вимірювання часу. Додатково створено замикання-логер, який зберігає префікс повідомлень. Для демонстрації порядку декораторів реалізовано два варіанти обгортання сервісної функції.

1. Створення структури проєкту

Проєкт розділено на модулі відповідно до їх призначення. У closures.py розміщено приклади замикань. У decorators.py реалізовано параметризовані декоратори. У

app.py створено сервісну функцію та демонстрацію композиції. Автоматизовані перевірки розміщено в каталозі tests.

Файл	Призначення
closures.py	Замикання make_logger() та make_counter() зі збереженням стану через nonlocal.
decorators.py	Параметризовані timed, retries і cached; формування ключів та інвалідація кешу.
app.py	Сервісна функція, два порядки композиції декораторів і консольна демонстрація.
tests/test_closures.py	Перевірки стану лічильника та роботи замикання-логера.
tests/test_decorators.py	Тести таймінгу, повторів, TTL, кеш-ключів, інвалідації та метаданих.
tests/test_composition.py	Перевірки впливу порядку декораторів і кеш-хітів.
README.md, ASSIGNMENT.md	Опис варіанта, реалізації та інструкцій запуску.
Requirements.txt, pyproject.toml, mypy.ini, .gitignore	Залежності й налаштування інструментів розробки.

Таблиця 1 — Структура проєкту

2. Реалізація замикань у файлі closures.py

Функція make_logger(prefix, sink) створює логер, який захоплює значення prefix із зовнішньої області видимості. Під час кожного виклику логер додає цей префікс до повідомлення та передає результат у sink. Завдяки ін’єкції sink повідомлення можна спрямовувати не лише в консоль, а й у список для тестування.

Функція make_counter(start) реалізує лічильник викликів. Внутрішня функція increment() змінює змінну value із зовнішньої функції за допомогою nonlocal. Після завершення make_counter() значення не зникає, а зберігається у замиканні між наступними викликами.

closures.py — замикання

```
"""Замикання для логування та підрахунку викликів."""
from collections.abc import Callable


def make_logger(prefix: str, sink: Callable[[str], None] = print) -> Callable[[str], None]:
    """Створює логер, який зберігає префікс у замиканні."""
    def log(message: str) -> None:
        sink(f"[{prefix}] {message}")
    return log


def make_counter(start: int = 0) -> Callable[[], int]:
    """Створює лічильник зі змінним станом, який зберігається в closure."""
    value = start

    def increment() -> int:
        nonlocal value
        value += 1
        return value

    return increment
```

Рисунок 1 — Реалізація замикань у файлі closures.py

3. Реалізація параметризованих декораторів

Декоратор timed() приймає мітку, одиницю вимірювання, логер і функцію годинника. Під час виклику wrapper() фіксується початковий час, виконується початкова функція, а в блоці finally обчислюється тривалість та надсилається повідомлення логеру. Блок finally гарантує логування навіть у випадку винятку.

Декоратор retries() приймає кількість спроб, типи винятків, початкову затримку, коефіцієнт backoff, джиттер, функцію sleep і логер. Після невдалої спроби обчислюється затримка delay × backoff^(attempt−1). Для тестування sleep замінюється функцією, яка лише записує передане значення, тому реального очікування немає.

decorators.py — timed та retries

```python
"""Параметризовані декоратори таймінгу, повторів і кешування."""
from __future__ import annotations

import functools
import random
import threading
import time
from collections.abc import Callable
from typing import Any, Hashable, ParamSpec, TypeVar, Literal

P = ParamSpec("P")
R = TypeVar("R")


def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None] = print,
    clock: Callable[[], float] = time.perf_counter,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Фабрика декоратора, що вимірює час виконання функції."""
    if unit not in ("s", "ms", "us"):
        raise ValueError("unit має бути 's', 'ms' або 'us'.")
    factor = {"s": 1.0, "ms": 1e3, "us": 1e6}[unit]

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            started = clock()
            try:
                return func(*args, **kwargs)
            finally:
                elapsed = (clock() - started) * factor
                logger(f"[timed] {name} took {elapsed:.3f} {unit}")
        return wrapper
    return decorator


def retries(
    attempts: int = 3,
    exceptions: tuple[type[BaseException], ...] = (Exception,),
    delay: float = 0.0,
    backoff: float = 1.0,
    jitter: float = 0.0,
    *,
    sleep: Callable[[float], None] = time.sleep,
    logger: Callable[[str], None] = lambda _message: None,
    random_fn: Callable[[], float] = random.random,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Повторює виклик після заданих винятків із backoff та jitter."""
    if attempts < 1:
        raise ValueError("attempts має бути не менше 1.")
    if delay < 0 or jitter < 0 or backoff < 1:
        raise ValueError("delay і jitter мають бути >= 0, backoff — >= 1.")

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            last_error: BaseException | None = None
            for attempt in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as error:
                    last_error = error
                    if attempt == attempts:
                        logger(f"[retries] failed after {attempt} attempts: {error!r}")
                        break
                    wait = delay * (backoff ** (attempt - 1))
                    if jitter:
                        wait += random_fn() * jitter
                    logger(f"[retries] attempt {attempt} failed; wait {wait:.3f}s")
                    sleep(wait)
```

Рисунок 2 — Реалізація декораторів таймінгу та повторів

Декоратор cached() формує ключ із позиційних і іменованих аргументів. Для зберігання значень використовується словник, а Lock захищає доступ до нього. Якщо TTL не задано, запис зберігається безстроково. Якщо час життя минув, запис видаляється під час наступного звернення. На обгортці доступні методи cache_clear() для повного очищення та cache_invalidate(predicate) для точкового видалення записів.

decorators.py — кеш із TTL та інвалідацією

```
            raise last_error
        return wrapper
    return decorator


def make_key(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Hashable:
    """Формує ключ кешу; аргументи мають бути хешованими."""
    try:
        return args, tuple(sorted(kwargs.items()))
    except TypeError as error:
        raise TypeError("Аргументи функції мають бути хешованими.") from error


def cached(
    *,
    ttl: float | None = None,
    time_fn: Callable[[], float] = time.time,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    """Кешує результати з необов'язковим TTL та інвалідацією записів."""
    if ttl is not None and ttl < 0:
        raise ValueError("ttl не може бути від'ємним.")

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        cache: dict[Hashable, tuple[float | None, R]] = {}
        lock = threading.Lock()

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            key = make_key(args, kwargs)
            now = time_fn()
            with lock:
                entry = cache.get(key)
                if entry is not None:
                    expires_at, value = entry
                    if expires_at is None or now < expires_at:
                        return value
                    del cache[key]
            value = func(*args, **kwargs)
            with lock:
                expires_at = None if ttl is None else time_fn() + ttl
                cache[key] = (expires_at, value)
            return value

        def cache_clear() -> None:
            with lock:
                cache.clear()

        def cache_invalidate(
            predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool]
        ) -> int:
            removed = 0
            with lock:
                for key in list(cache):
                    args_key, kwargs_key = key
                    if predicate(args_key, dict(kwargs_key)):
                        del cache[key]
                        removed += 1
            return removed

        setattr(wrapper, "cache_clear", cache_clear)
        setattr(wrapper, "cache_invalidate", cache_invalidate)
        return wrapper
    return decorator
```

Рисунок 3 — Кешування з TTL та інвалідацією

4. Композиція декораторів у файлі app.py

У демонстрації створено сервісну функцію unstable_fetch(user_id), яка імітує нестабільний зовнішній сервіс. Перші два звернення для заданого користувача завершуються TimeoutError, а третє повертає результат. Реальна мережа не використовується. Лічильник звернень створено через make_counter(), а повідомлення передаються через closure-логер.

Перший порядок, cache_outer, відповідає композиції cached(timed(retries(function))). Кеш є зовнішнім шаром. Після першого успішного виконання результат зберігається, а повторний виклик із тими самими аргументами повертається без запуску таймінгу та повторів. Отже, у логах є лише одне повідомлення timed для двох викликів сервісної обгортки.

Другий порядок, retry_outer, відповідає композиції timed(retries(cached(function))). Тут таймінг є зовнішнім шаром, а кеш розташований усередині повторів. Помилки не кешуються, оскільки результат зберігається лише після успішного виконання. Повторний виклик проходить через зовнішній таймер і внутрішній шар повторів, але отримує результат із кешу. Тому для двох звернень формується два повідомлення timed.

app.py — композиція декораторів

```python
"""Демонстрація композиції декораторів для сервісної функції."""
from __future__ import annotations

from closures import make_counter, make_logger
from decorators import cached, retries, timed


def build_service(order: str = "cache_outer"):
    """Створює сервіс із заданим порядком декораторів і спільним станом."""
    logs: list[str] = []
    logger = make_logger("SERVICE", logs.append)
    calls = make_counter()
    attempts_by_user: dict[int, int] = {}

    def unstable_fetch(user_id: int) -> dict[str, object]:
        calls()
        attempts_by_user[user_id] = attempts_by_user.get(user_id, 0) + 1
        if attempts_by_user[user_id] < 3:
            raise TimeoutError(f"тимчасова помилка для користувача {user_id}")
        return {"user_id": user_id, "name": f"User {user_id}"}

    retry_layer = retries(
        attempts=3, delay=0.1, backoff=2.0, jitter=0.0,
        sleep=lambda _seconds: None, logger=logger,
    )(unstable_fetch)
    timed_retry = timed("service", unit="ms", logger=logger, clock=_fake_clock())(retry_layer)

    if order == "cache_outer":
        service = cached(ttl=30.0)(timed_retry)
    elif order == "retry_outer":
        service = timed(
            "service", unit="ms", logger=logger, clock=_fake_clock()
        )(retries(
            attempts=3, delay=0.1, backoff=2.0, sleep=lambda _seconds: None,
            logger=logger,
        )(cached(ttl=30.0)(unstable_fetch)))
    else:
        raise ValueError("order має бути 'cache_outer' або 'retry_outer'.")

    return service, calls, logs


def _fake_clock():
    """Повертає годинник із детермінованим кроком для демонстраційного логу."""
    ticks = iter(i * 0.001 for i in range(1000))
    return lambda: next(ticks)


def main() -> None:
    print("=== Лабораторна робота 6 — варіант 15 ===")
    for order in ("cache_outer", "retry_outer"):
        service, calls, logs = build_service(order)
        print(f"\nПорядок: {order}")
        print("Перший виклик:", service(7))
        print("Повторний виклик:", service(7))
        print("Кількість звернень до сервісної функції:", calls() - 1)
        print("Лог:")
        for line in logs:
            print(" ", line)
        print("Очищення кешу та повторний виклик:")
        clear = getattr(service, "cache_clear", None)
        if clear is not None:
            clear()
        print(service(7))


if __name__ == "__main__":
    main()
```

Рисунок 4 — Демонстрація композиції декораторів у app.py

У Python декоратори застосовуються знизу вгору. Тобто запис із кількома декораторами еквівалентний послідовному вкладенню викликів функцій. Саме зовнішнє положення кешу або таймера визначає, які шари будуть пропущені під час кеш-хіту.

5. Створення та виконання тестів

Для перевірки реалізації створено автоматизовані тести pytest у трьох файлах. Вони перевіряють такі властивості:

лічильник зберігає стан між викликами, а логер захоплює префікс із зовнішньої області;

timed використовує ін’єктований clock і logger, зберігає __name__, __doc__ та __wrapped__, а також записує час у разі винятку;

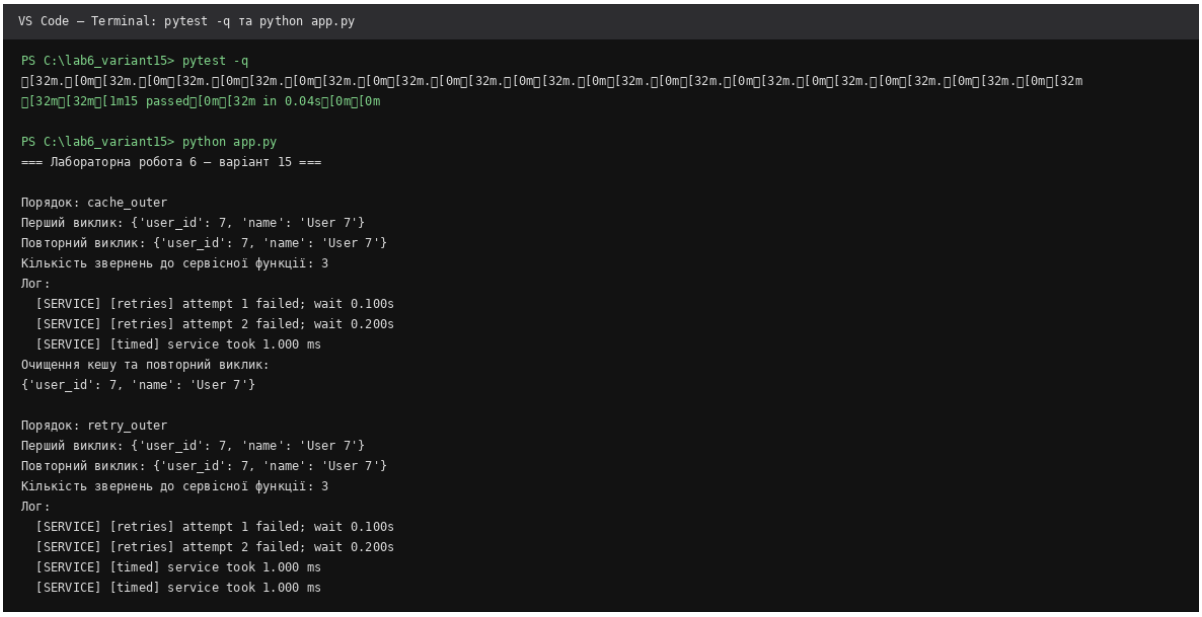
retries виконує повтори до успіху, повторно піднімає виняток після останньої спроби та використовує fake sleep без реальної затримки;

cached повертає значення з кешу, враховує TTL, формує ключі з args і kwargs, підтримує точкову та повну інвалідацію;

обидва порядки композиції повертають правильний результат, повторний виклик використовує кеш, а кількість звернень до сервісної функції не збільшується;

некоректні параметри attempts, delay і ttl відхиляються винятком ValueError.

Під час контрольного запуску pytest усі 15 тестів завершилися успішно. Тести не використовують реальний sleep: затримки підмінено функцією, а годинник і час кешу передаються через ін’єкцію.


VS Code — Terminal: pytest -q та python app.py
PS C:\lab6_variant15> pytest -q
[32m.[0m[32m.[0m[32m.[0m

Для порядку cache_outer таймер спрацьовує один раз, оскільки другий виклик завершується на зовнішньому рівні кешу. Для порядку retry_outer таймер спрацьовує двічі, оскільки він охоплює весь ланцюжок і запускається при кожному зверненні. При цьому друге звернення не виконує повторних спроб сервісної операції, а повертає кешований результат.

7. Перевірка стилю та типів

У requirements.txt зазначено pytest, mypy, black і ruff. У pyproject.toml задано довжину рядка 88 символів і цільову версію Python 3.11. У mypy.ini увімкнено строгий режим перевірки типів. Для локальної перевірки передбачено команди:

```
pytest -q
python app.py
mypy .
black --check .
ruff check .
```

Під час підготовки звіту підтверджено успішне проходження 15 тестів pytest та запуск демонстраційної програми. Перевірки mypy, Black і Ruff необхідно виконати локально після встановлення відповідних пакетів; їх проходження в цьому звіті не позначається як підтверджене.

Висновок:

У ході лабораторної роботи я реалізувала 15-й варіант — композицію декораторів для сервісної функції. Я створила два замикання: логер із захопленим префіксом і лічильник зі змінним станом через nonlocal. Також я реалізувала параметризовані декоратори timed, retries та cached із використанням functools.wraps, ParamSpec і TypeVar.

Я дослідила два порядки композиції декораторів. У випадку зовнішнього кешу повторний виклик пропускає таймер і механізм повторів. Якщо кеш розташований усередині повторів, зовнішній таймер запускається для кожного звернення, але успішний результат усе одно повертається з кешу. Ін’єкція clock, sleep, logger і time_fn дала змогу виконати детерміновані перевірки без реальних затримок.

Автоматизовані тести pytest (15 тестів) підтвердили коректність замикань, повторних спроб, таймінгу, кешування, TTL, інвалідації та збереження метаданих функцій. Таким чином, я закріпила навички роботи із замиканнями, функціями вищого порядку та параметризованими декораторами й навчилася оцінювати вплив їх порядку на виконання програми.